

1 Overview

We will replicate the Virus model, which simulates the transmission and persistence of a directly transmitted virus in a human population. It implements an agent-based version of the classical SIR (Susceptible-Infected-Recovered) framework introduced in week 8's lecture, capturing how population-level behaviour emerges from local interactions between individual agents with no central control, rather than from a system of differential equations. This makes it relevant for students and researchers studying such interactions.

2 Design of Existing Model

Each tick (one week), the model simulates a population of people (150 by default, with 10 initially infected) on a 2D (35x35) patch grid.

Each person is in one of three states:

- *Susceptible (Green, healthy but can catch the virus)*
- *Infected (Red, sick and infectious)*
- *Immune (Grey, recovered and temporarily protected)*

At every tick each person does the following:

1. *Age by one week, decrement immunity counter, advance sick-time; die of old age at 50 years*
 - a. *Immunity expires after 52 weeks; the person returns to susceptible*
2. *Move: turn by a small random angle, then move forward one unit*
3. *If sick and the duration has elapsed, recover with probability chance-recover (becoming immune) or die*
4. *If still sick, attempt to infect each susceptible person on the same patch with probability infectiousness; otherwise (if healthy), reproduce with probability 1% while population is below the carrying capacity of 300, offspring are susceptible*

User-controlled parameters: infectiousness, chance-recover, duration, number-people.

Outputs: per-tick counts of susceptible, infected, and immune individuals, plus total population.

3 Design of Our Model

We will reimplement the model in Python as the following classes:

Class	Key Attributes	Key Methods	Description
Person	state, age_weeks, sick_weeks, immune_weeks, x, y, heading	get_older(), move(), recover_or_die(), try_infect_neighbours(), try_reproduce()	Agent class: Tracks state and defines per-tick behaviours
World	params, rng, current_tick, population, patches	tick(), people_on_patch(x, y), update_patch(p, old, new), count_by_state(s)	Owns the grid, population, and RNG; drives per-tick simulation
State	SUSCEPTIBLE, INFECTED, IMMUNE	-	3 Health states
Params	infectiousness,	load(path),	Loads experiment

	chance_recover, duration, number_people, carrying_capacity, lifespan_weeks, immunity_duration, birth_rate, grid_width, grid_height, max_ticks, random_seed, extension_enabled, vaccination_rate	with_seed(seed)	parameters from .ini file
Stats	output_path	record_tick(world), close()	Records metrics; writes CSV
Main	-	main(argv)	Entry point; runs the simulation loop

The per-tick lifecycle described in Section 2 is implemented in `World.tick()`, which iterates over the shuffled population and calls each Person's `get_older()`, `move()`, `recover_or_die()`, and either `try_infect_neighbours()` or `try_reproduce()` in that order, mirroring the NetLogo `go` procedure.

Configuration is done via `.ini` files parsed by `Params`, including the random seed for reproducibility.

4 Extension and Research Question

Research Question: What vaccination coverage is required to drive the virus to extinction, and how does this depend on infectiousness?

Design: A configurable fraction of the initial population is set to IMMUNE at setup, representing vaccinated individuals. The extension is purely additive: with `vaccination_rate = 0`, it reduces to the base model.

Experiment: Section 5, E1.

5 Experiments

Replication experiments:

#	Sweep	Purpose
1	Default parameters	Sanity check; both should reach a steady state
2	infectiousness $\in \{10, 30, 50, 70, 90\}\%$	Locate extinction-persistence threshold
3	chance-recover $\in \{25, 50, 75, 95\}\%$	How recovery rate affects virus lethality
4	duration $\in \{5, 10, 20, 40\}$ weeks	How sickness duration affects virus persistence
5	Ebola and HIV parameter calibrations	Qualitative comparison of real-world cases as suggested on NetLogo
E1	<code>vaccination_rate</code> x infectiousness across a range of values	Find the vaccination threshold above which the virus consistently goes extinct

SWEN90004: A2 Project Proposal

Team Member: Luka Apps, Ka Chun Lee, Will Kang

For each cell we will run 30 seeds in both Python and NetLogo and compare extinction probability, peak prevalence, time-to-peak, and steady-state prevalence (the latter three conditional on outbreak) using means with 95% confidence intervals and two-sample t-tests. Outcome distributions will be inspected for bimodality before pooling.

6 Task Distribution

All three group members contribute across the project, with each taking primary ownership of one stream as shown below. We aim to finish the simulation code early so that the bulk of our remaining time can be spent on experiments and the report, which we expect to be the most substantial portion of the work.

Week	Luka Apps	Ka Chun Lee	Will Kang
W9 (Proposal)	NetLogo exploration	Python skeleton & class design	Background reading & proposal lead
W10	World class & tick loop	Person class & state transitions	Params, Stats, Main & batch script
W11	Replication experiments (Python)	Replication experiments (NetLogo BehaviorSpace)	Extension implementation & analysis script
W12	Report: model design & implementation	Report: results & validation	Report: extension, discussion & final edits